

Grille d'audit croisé — Contrat OpenAPI

Commission d'architecture — Séance 9

Groupe auditeur	Contrat audité	Créneau
ThermoSense	ThermoSense	20 min · restitution 11h45

Chaque groupe réalise 2 audits sur la journée (20 min chacun). Vous ne remettez qu'une seule fiche avant 11h45 — celle qui produit le retour le plus solide. La restitution orale porte sur cette fiche retenue. ½ page de rendu — soyez concis et factuels.

Grille d'évaluation rapide

Pour chaque critère, cochez le niveau observé et notez un commentaire factuel (1-2 lignes max).

Critère 1 — Endpoint testable sans lire le code

Peut-on construire une requête valide sur au moins un endpoint en lisant uniquement le contrat ?

- ☒ **Oui** — Le contrat suffit (URL, méthode, paramètres, authentification)
- ☐ **Partiel** — Des informations manquent (ex. format d'authentification non précisé)
- ☐ **Non** — Impossible sans aide extérieure

Commentaire : POST /auth/login est bon : corps LoginRequest documenté, réponse AuthResponse avec token JWT. Le Bearer JWT est défini et appliqué globalement — on peut tester immédiatement sans consulter le code.

Critère 2 — Erreurs 4xx et 5xx documentées

Les réponses d'erreur sont-elles décrites (code HTTP, corps de réponse, signification) ?

- ☐ **Oui** — Au moins les principales erreurs sont documentées avec corps structuré
- ☒ **Partiel** — Seulement certains endpoints ou seulement les codes sans corps
- ☐ **Non** — Uniquement les cas 200/201 documentés

Commentaire : Les endpoints métier (building, zone, sensor...) documentent 400/404/500 avec le schéma Error structuré (code + message) et 429 avec headers. En revanche, /auth/register (400) et /auth/login (401) n'ont pas de corps de réponse défini.

Critère 3 — Stratégie de versioning visible

Peut-on identifier la version de l'API et comprendre la stratégie adoptée ?

- ☐ **Oui** — Versioning visible (URL, header) et expliqué
- ☒ **Partiel** — Version présente mais stratégie non explicitée
- ☐ **Non** — Aucun versioning identifiable

Commentaire : La version 1.0.0 est déclarée dans info.version mais aucun préfixe /v1/ n'apparaît dans les paths, et aucune stratégie d'évolution (dépréciation, header Accept-Version...) n'est documentée.

Critère 4 — Exemples présents

Y a-t-il des exemples de requête et/ou de réponse dans le contrat ?

- ☐ **Oui** — Exemples présents pour les endpoints principaux
- ☐ **Partiel** — Quelques exemples, non systématiques
- ☒ **Non** — Aucun exemple

Commentaire : Aucun champ exemple ni bloc exemples n'est présent dans les schémas ou les réponses. Le README donne un exemple de payload pour /actuator/{id}/command mais celui-ci n'est pas intégré au contrat OpenAPI.

Critère 5 — Cohérence des schemas

Les types, formats et contraintes (required, type, format, enum) sont-ils définis de façon cohérente ?

- ☐ **Oui** — Schemas complets et cohérents entre endpoints
- ☒ **Partiel** — Quelques schemas incomplets ou incohérents
- ☐ **Non** — Schemas absents ou très lacunaires

Commentaire : Sensor.type et AlertThreshold.type n'ont pas d'enum. GenericRequest (eTag, idempotencyKey) est défini en composant mais n'est référencé par aucun endpoint. Le body de PATCH /zone/{id} et PATCH /building/{id} réutilise le schéma complet (incluant id/createdAt en lecture seule), sans schéma dédié au PATCH.

Critère 6 — README opérationnel

Le README permet-il de démarrer le projet sans contacter l'équipe ?

- ☒ **Oui** — Variables d'environnement, commandes de démarrage, prérequis couverts
- ☐ **Partiel** — L'essentiel est là mais des étapes sont implicites
- ☐ **Non** — README absent, vide, ou insuffisant pour démarrer

Commentaire : make init + make up suffisent pour démarrer. Les variables d'env sont dans .env.example, les commandes Prisma, la seed et Swagger UI sont documentés. Les décisions d'architecture BFLA/BOLA sont expliquées avec tableau de justification.

Critère 7 — Cohérence endpoints / ressources

Les endpoints exposés forment-ils un ensemble cohérent ? (nommage uniforme, logique REST respectée)

- ☐ **Oui** — Nommage et structure cohérents, verbes HTTP appropriés
- ☒ **Partiel** — Quelques incohérences isolées
- ☐ **Non** — Incohérences significatives (nommage mixte, verbes mal utilisés, structure cassée)

Commentaire : Les paths sont en anglais mais les tags OpenAPI mêlent français (Bâtiment, capteur, actionneur) et anglais (measurement) avec une casse incohérente. GET /alert-threshold/{id} est absent alors que les autres ressources exposent toutes un GET par ID. La création passe par le parent (/zone/{id}/sensor) de façon cohérente.

Critère 8 — Sécurité visible dans le contrat

Les endpoints sensibles documentent-ils l'authentification requise (securitySchemes, security) ?

- ☐ **Oui** — Schéma de sécurité défini et appliqué aux endpoints concernés
- ☒ **Partiel** — Présent pour certains endpoints seulement
- ☐ **Non** — Aucune information de sécurité dans le contrat

Commentaire : Le bearerAuth JWT est bien défini et appliqué globalement (security: [bearerAuth]).

Problème : /auth/register et /auth/login n'écrasent pas security[]: le contrat implique qu'un token est requis pour s'enregistrer/se connecter, ce qui est contradictoire.

Synthèse

Point fort identifié

Ce que ce contrat fait bien — soyez précis

les décisions d'architecture sécurité (BFLA/BOLA) sont justifiées sous forme de tableau question/réponse, avec les scénarios adverses et nominaux couverts, et les preuves automatiques référencées Le rate limiting sur PATCH /actuator/{id} est documenté avec le seuil, la dimension de comptage et le comportement en cas de dépassement

Incohérence ou manque identifié

1 point concret, avec référence à l'endpoint ou à la section concernée

Sécurité contradictoire sur les endpoints d'authentification (openapi.yml, paths /auth/register et /auth/login) : la clé security: [bearerAuth] est appliquée globalement sans override sur ces deux endpoints. Le contrat indique donc qu'un JWT valide est requis pour s'enregistrer et se connecter — ce qui est impossible par définition.

Question ouverte pour le groupe audité

Ce que vous aimeriez leur demander pendant la restitution

Le champ type de Sensor et de AlertThreshold n'a pas d'enum dans le contrat, alors que Actuator.type et Actuator.state en ont. Était-ce intentionnel pour laisser de la flexibilité au client, ou y a-t-il une liste de valeurs supportées en pratique (température, humidité, CO₂...) que vous n'avez pas encore formalisée ?

Consignes de restitution (11h45, 5 min max par groupe)

Structure imposée pour votre restitution orale :

1. **1 point fort** : ce que ce contrat fait mieux que la moyenne — avec un exemple précis.
2. **1 incohérence ou manque** : ce que vous avez trouvé, avec la référence dans le contrat.
3. **1 question ouverte** : posée directement au groupe audité (ils ont 1 minute pour répondre).

Rappel : L'audit est formatif. L'objectif n'est pas de noter, mais de donner un retour utile. Soyez factuels, pas rhétoriques.